

卷积神经网络LeNet实战MNIST手写数字识别

1.深度学习工程的结构

这里首先介绍如何建立一个合理的PyTorch深度学习工程。

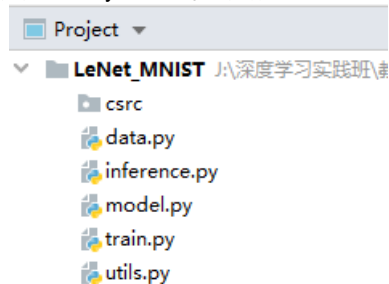
由于Python的模块化特性，我们可以很方便地把一个深度学习模型拆分成几个文件，每个文件负责深度学习工程的一小部分，包括深度学习模型包含的类和模块、深度学习模型的训练和测试部分、数据的预处理等。

同时因为拆分成多个独立的文件，也方便对这些文件包含的部分进行独立测试。

在Python中可以使用import语句来导入包的特性，可以在不同的任务（训练 / 测试）中使用同一个模型，以减少不同代码之间的耦合，使代码更清晰。

以下是常用的一种深度学习模型的工程实践，大家可以参考这个工程实践，在学习和工作中对模型的实践进行自我总结，并建立一个自己觉得方便的工程实践。

本项目实践案例LeNet_MNIST相关的文件排列方式如PyCharm中打开所示：



工程文件的排列代码可以参考以下代码：

```
LeNet_MNIST/  
  csrc/  
  data.py  
  inference.py  
  model.py  
  train.py  
  utils.py
```

代码中，LeNet_MNIST代表的是工程目录。在这个目录下的文件中，

- data.py,这个文件负责数据载入的模块，包含和数据载入相关的Dataset类和DataLoader类；
- utils.py,包含深度学习工程中常用的一些工具，比如，对于模型的模块和参数的一些预处理等；
- inference.py代表模型预测（推断）部分的代码；
- model.py,这也是模型的核心部分，所有的模型中包含的模块和子模块一般都在这个文件中定义；
- train.py,代表模型训练部分的代码；
- csrc目录，由于PyTorch代码中可以引入C/C++语言编写的模块，加速模块的计算速度，或者实现一些PyTorch没有的模块，因此，在一些情况下可以引入C/C++语言编写的代码，并且使用pybind11库导出供PyTorch使用，在这种情况下，csrc目录就可以用来放置对应的代码，编译后的动态链接库可以导出为接口，以包的形式提供给Python导入和调用。

【注意】以上方式只是一种文件排列方式，大家可以根据自己的个人喜好来重命名和重新组织所有的工程文件，但是最好能够对整个工程进行模块化，并且按照模块的功能来组织文件，避免把所有的子模块都堆砌在一起的现象发生，否则将不利于工程代码的修改和调试。

2.MNIST数据集的准备

下面将从构造一个LeNet网络开始，来演示如何构建一个卷积神经网络模型进行手写数字的识别。

首先是MNIST数据集的载入。

LeNet神经网络的输入是32x32,因此，这里首先需要把MNIST的输入28x28进行缩放，使其大小为32x32。

```
In [1]: # 该段代码用于LeNet的训练数据集MNIST的载入

from torchvision.datasets import MNIST
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

data_train = MNIST('./data',
                   download=True,
                   transform=transforms.Compose([
                       transforms.Resize((32, 32)),
                       transforms.ToTensor()]))
data_test = MNIST('./data',
                  train=False,
                  download=True,
                  transform=transforms.Compose([
                       transforms.Resize((32, 32)),
                       transforms.ToTensor()]))
```

数据集下载过程截图如下：

```
Downloading http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz to ./data\MNIST\raw\train-images-idx3-ubyte.gz
9920512it [00:10, 990248.62it/s]

Extracting ./data\MNIST\raw\train-images-idx3-ubyte.gz to ./data\MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz to ./data\MNIST\raw\train-labels-idx1-ubyte.gz
32768it [00:01, 17532.36it/s]

Extracting ./data\MNIST\raw\train-labels-idx1-ubyte.gz to ./data\MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz to ./data\MNIST\raw\t10k-images-idx3-ubyte.gz
1654784it [00:09, 182090.67it/s]

Extracting ./data\MNIST\raw\t10k-images-idx3-ubyte.gz to ./data\MNIST\raw
Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz to ./data\MNIST\raw\t10k-labels-idx1-ubyte.gz
8192it [00:15, 514.98it/s]

Extracting ./data\MNIST\raw\t10k-labels-idx1-ubyte.gz to ./data\MNIST\raw
Processing...
Done!
```

```
In [2]: data_train_loader = DataLoader(data_train, batch_size=256,
    shuffle=True, num_workers=2)
data_test_loader = DataLoader(data_test, batch_size=1024,
    num_workers=2)
```

在代码中构建了两个数据集data_train和data_test,分别代表MNIST的训练数据集和测试数据集。

由于设置了download=True,数据集会被下载到当前目录下的data文件夹下,同时,如果需要取出数据集中的某一幅图像,则会对图像做大小变换transforms.Resize,让输出图像的大小符合32x32的大小,最后将图像转变为浮点张量并输出。

通过使用DataLoader载入对应的训练数据集和测试数据集的实例,在上述代码最后获得了两个数据集的载入器:

- **data_train_loader**数据集的载入器载入的数据迷你批次大小为256,会对数据集中的图像做随机排列 (shuffle=True),同时使用2个并行的进程对数据进行处理,最后输出的结果是256x1x32x32大小的张量,其中通道数为1,代表输出的图像张量是只有灰度色阶的黑白图像;
- **data_test_loader**数据集为测试数据集,因为在测试数据集中运行模型并不需要记录计算图以及计算反向传播梯度,节约了内存。因此,在代码中相应的迷你批次的大小也有了增加,这里设置的迷你批次的大小为1024,同时工作的进程数目为2.对于测试数据集,因为只是为了测试,并不会出现每次输入数据顺序一致而导致模型过拟合的情况发生,所以这里的输入图像不会做随机排列 (默认shuffle=False)。

上述数据载入代码可以放在工程文件的代码data.py中,也可以分成训练数据和测试数据两部分,分别放入 train.py和inference.py中。

有了数据集之后,大家可以使用Python的for...in...语法来测试数据集的输出,并且使用一些画图工具 (如Matplotlib)对MNIST数据集的输出绘图,检查输出的图像是否正确。

这里仅仅演示data_train_loader的部分输出结果,代码存放在工程文件utils.py中。MNIST数据展示代码如下:

```
In [3]: import matplotlib.pyplot as plt
%matplotlib inline
figure = plt.figure()
num_of_images = 60

for imgs, targets in data_train_loader:
    break

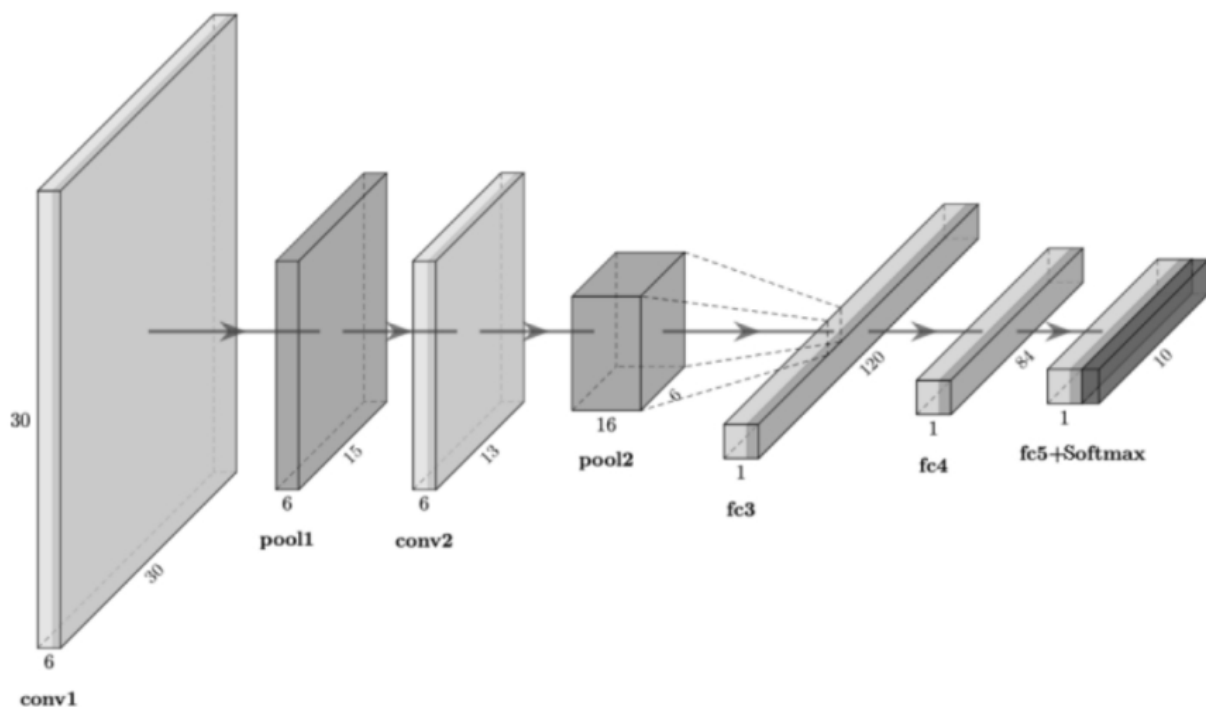
for index in range(num_of_images):
    plt.subplot(6, 10, index+1)
    plt.axis('off')
    img = imgs[index, ...]
    plt.imshow(img.numpy().squeeze(), cmap='gray_r')
plt.show()
```

```
6 4 1 3 9 9 6 6 4 3
7 9 9 9 0 4 2 7 5 9
4 8 3 6 0 1 9 4 8 6
0 7 8 0 6 1 9 2 1 6
6 1 2 1 3 4 7 7 8 7
4 1 6 8 9 4 6 2 7 0
```

3.LeNet网络的搭建

有数据集的预处理代码之后，接下来需要着手构建LeNet模型。

LeNet模型的结构如下图所示。



可以看到，整个神经网络一共有6个组成部分：

- conv1，其构成3x3的卷积，输出的特征有6个通道，空间大小为30x30；
- pool1，是2x2的步长为2的池化pool1,输出空间大小为15x15的特征；

- conv2,同样是一个3x3的卷积，输出的特征有16个通道，空间大小为13x13；
- pool2，同样是2x2的步长为2的池化pool2,输出特征的空间大小为6x6；
- fc3，把四维张量转换为二维张量，其中CHW三个维度被合并到一个维度，使用fc3进行线性变换，输出新的特征数目为120的二维张量；
- fc4，使用fc4进行线性变换，输出特征数目为84的二维张量，
- fc5+Softmax，使用fc5做线性映射，映射到10个分类，如果需要取得具体分类的概率，则使用Softmax激活函数作用于第1个维度（维度从0开始编号），得到每个分类的概率。

LeNet神经网络模型的代码如下：

```
In [4]: # 该代码定义了LeNet模型

import torch
import torch.nn as nn

class LeNet(nn.Module):

    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 3)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 3)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc3 = nn.Linear(16*6*6, 120)
        self.fc4 = nn.Linear(120, 84)
        self.fc5 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool1(torch.relu(self.conv1(x)))
        x = self.pool2(torch.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = torch.relu(self.fc3(x))
        x = torch.relu(self.fc4(x))
        x = self.fc5(x)
        return x
```

可以看到，这里定义了对应的卷积层conv1和conv2,池化层pool1、pool2,以及全连接层fc3、fc4、fc5。

通过一系列的卷积和线性变换，最后输出Softmax之前的神经网络的值（这个值也称为Logits),这个张量（Logits)可以作为输入张量直接输入到交叉熵的损失函数类torch.nn.CrossEntropyLoss来计算模型最后输出的损失函数。

在构建模型的类以后，可以通过输入随机的张量来测试模型的输出结果。

具体使用的代码如下：

```
In [5]: model = LeNet() # 初始化实例
ret = model(torch.randn(1, 1, 32, 32)) # 输入一张图片，测试输出结果
ret.shape # torch.Size([1, 10])
```

```
Out[5]: torch.Size([1, 10])
```

可以看到，在测试代码中，

- 首先初始化了一个LeNet的神经网络实例，
- 然后输入随机张量，这个张量的大小为1x1x32x32,代表输入的是一张32x32大小的黑白图像，
- 最后可以见到输出的结果是1x10,如果使用Softmax函数对张量在第一个维度上进行作用，可以得到每个模型预测的该图像对应的每个数字的概率。

神经网络的简单测试代码可以集成到 model.py文件中，利用Python库函数的特性，可以做到只有在执行 model.py脚本的时候才会测试这个代码。

4. LeNet网络的训练和测试

在完成模型的构建代码后，下一步需要做的是构建训练和测试代码，这部分代码可以放在train.py文件中，具体代码如下：

In [6]: # 该代码定义了LeNet模型的训练过程

```
import torch
import torch.nn as nn
# from model import LeNet #若放入工程文件时，需要使用此命令
import torch.optim as optim

# ... 此处略去定义训练数据载入器的代码，具体参考代码data.py

model = LeNet() # 定义LeNet模型
model.train() # 切换模型到训练状态
lr = 0.01 # 定义学习率
criterion = nn.CrossEntropyLoss() # 定义损失函数
optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9,
                        weight_decay=5e-4) # 定义随机梯度下降优化器

train_loss = 0
correct = 0
total = 0

for batch_idx, (inputs, targets) in enumerate(data_train_loader):

    optimizer.zero_grad()
    outputs = model(inputs)
    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()

    train_loss += loss.item()
    _, predicted = outputs.max(1)
    total += targets.size(0)
    correct += predicted.eq(targets).sum().item()

    if batch_idx % 50 == 0 or batch_idx == (len(data_train_loader)-1) :
        print(batch_idx, len(data_train_loader), 'Loss: %.3f | Acc: %.3f%% (%d/%d)'
              % (train_loss/(batch_idx+1), 100.*correct/total, correct, total))
```

```
0 235 Loss: 2.305 | Acc: 8.984% (23/256)
50 235 Loss: 2.295 | Acc: 13.166% (1719/13056)
100 235 Loss: 2.268 | Acc: 27.576% (7130/25856)
150 235 Loss: 1.934 | Acc: 39.691% (15343/38656)
200 235 Loss: 1.575 | Acc: 50.871% (26176/51456)
234 235 Loss: 1.398 | Acc: 56.323% (33794/60000)
```

在上述代码中，首先使用LeNet类初始化模型的实例，构造LeNet模型的一个实例，这里用model来表示。

同时设置学习率为0.01(用lr来表示)，以及对应的交叉熵损失函数（用criterion来表示）。

接下来定义模型的优化器，这里定义的是随机梯度下降优化器。

然后是迭代训练数据载入器，得到每一个迷你批次的输入图像数据inputs和预测目标targets,计算每一迷你批次的损失函数，对损失函数做反向传播计算，得到参数的梯度，接着使用随机梯度下降优化器对模型参数进行优化。

最后则是模型数据的输出过程，包括计算平均损失函数和平均准确率。

需要注意的是，**这里的训练代码仅仅是一个迭代期的（Epoch）训练过程**，对于多个迭代期的训练过程，需要在for循环迭代迷你批次的数据的外侧再添加一个for循环，用于进行多个迭代期的训练。

可以将模型保存，代码如下：

In [7]: # 模型的保存
torch.save(model.state_dict(), 'model_lenet.pth') # 模型的状态字典

训练代码包含迭代期的（Epoch）训练过程：

In [8]: # 该代码定义了LeNet模型的训练过程，加入epoch

```
import torch
import torch.nn as nn
# from model import LeNet #若放入工程文件时，需要使用此命令
import torch.optim as optim

# ... 此处略去定义训练数据载入器的代码，具体参考代码data.py

model = LeNet() # 定义LeNet模型
model.train() # 切换模型到训练状态
lr = 0.01 # 定义学习率
epochs = 4 #定义训练轮次
criterion = nn.CrossEntropyLoss() # 定义损失函数
optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9,
                        weight_decay=5e-4) # 定义随机梯度下降优化器

for epoch in range(epochs):
    correct = 0
    total = 0
    train_loss = 0
    for batch_idx, (inputs, targets) in enumerate(data_train_loader):

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += predicted.eq(targets).sum().item()

        if batch_idx % 50 == 0 or batch_idx == (len(data_train_loader)-1) :
            print(epoch, batch_idx, len(data_train_loader), 'Loss: %.3f | Acc: %.3f%% (%d/%d)'
                  % (train_loss/(batch_idx+1), 100.*correct/total, correct, total))

# 模型的保存
torch.save(model.state_dict(), 'model_lenet_epoch.pth') # 模型的状态字典
```

```
0 0 235 Loss: 2.302 | Acc: 10.547% (27/256)
0 50 235 Loss: 2.300 | Acc: 10.968% (1432/13056)
0 100 235 Loss: 2.296 | Acc: 13.413% (3468/25856)
0 150 235 Loss: 2.277 | Acc: 20.788% (8036/38656)
0 200 235 Loss: 2.020 | Acc: 32.086% (16510/51456)
0 234 235 Loss: 1.801 | Acc: 39.517% (23710/60000)
1 0 235 Loss: 0.395 | Acc: 87.891% (225/256)
1 50 235 Loss: 0.402 | Acc: 87.538% (11429/13056)
1 100 235 Loss: 0.378 | Acc: 88.420% (22862/25856)
1 150 235 Loss: 0.345 | Acc: 89.298% (34519/38656)
1 200 235 Loss: 0.326 | Acc: 89.908% (46263/51456)
1 234 235 Loss: 0.316 | Acc: 90.227% (54136/60000)
2 0 235 Loss: 0.211 | Acc: 93.359% (239/256)
2 50 235 Loss: 0.218 | Acc: 93.536% (12212/13056)
2 100 235 Loss: 0.204 | Acc: 93.897% (24278/25856)
2 150 235 Loss: 0.198 | Acc: 93.970% (36325/38656)
2 200 235 Loss: 0.192 | Acc: 94.141% (48441/51456)
2 234 235 Loss: 0.189 | Acc: 94.198% (56519/60000)
3 0 235 Loss: 0.143 | Acc: 96.484% (247/256)
3 50 235 Loss: 0.150 | Acc: 95.259% (12437/13056)
3 100 235 Loss: 0.143 | Acc: 95.510% (24695/25856)
3 150 235 Loss: 0.138 | Acc: 95.667% (36981/38656)
3 200 235 Loss: 0.133 | Acc: 95.841% (49316/51456)
3 234 235 Loss: 0.130 | Acc: 95.920% (57552/60000)
```

最后则是模型的测试模块。

首先需要载入保存的模型。

推断的代码在工程文件中命名为inference.py，具体代码如下：

In [9]: # 该代码定义了LeNet模型的推断过程

```
import torch
import torch.nn as nn
# from model import LeNet #若放入工程文件时，需要使用此命令

# ... 此处略去定义测试数据载入器的代码，具体参考代码data.py

# 假设模型保存在model_lenet.pth文件中，也可选择多迭代期训练的模型model_lenet_epoch.pth
model_path = "/model_lenet.pth"
model = LeNet() # 定义LeNet模型
criterion = nn.CrossEntropyLoss() # 定义损失函数
model.load_state_dict(torch.load(model_path)) # 载入模型和参数
model.eval() # 切换模型到测试状态

test_loss = 0
correct = 0
total = 0
with torch.no_grad(): # 关闭计算图
    for batch_idx, (inputs, targets) in enumerate(data_test_loader):

        outputs = model(inputs)
        loss = criterion(outputs, targets)

        test_loss += loss.item()
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += predicted.eq(targets).sum().item()

    print(batch_idx, len(data_test_loader), 'Loss: %.3f | Acc: %.3f%% (%d/%d)'
          % (test_loss/(batch_idx+1), 100.*correct/total, correct, total))
```

```
0 10 Loss: 0.329 | Acc: 89.062% (912/1024)
1 10 Loss: 0.385 | Acc: 87.891% (1800/2048)
2 10 Loss: 0.390 | Acc: 87.500% (2688/3072)
3 10 Loss: 0.400 | Acc: 87.183% (3571/4096)
4 10 Loss: 0.397 | Acc: 87.109% (4460/5120)
5 10 Loss: 0.373 | Acc: 87.891% (5400/6144)
6 10 Loss: 0.356 | Acc: 88.491% (6343/7168)
7 10 Loss: 0.344 | Acc: 88.892% (7282/8192)
8 10 Loss: 0.325 | Acc: 89.507% (8249/9216)
9 10 Loss: 0.326 | Acc: 89.630% (8963/10000)
```

In [10]: # 该代码定义了LeNet模型的推断过程

```
import torch
import torch.nn as nn
# from model import LeNet 若LeNet放入工程文件model.py时，需要使用此命令

# ... 此处略去定义测试数据载入器的代码，具体参考代码data.py

# 假设模型保存在model_lenet.pth文件中，也可选择多迭代期训练的模型model_lenet_epoch.pth
model_path = "./model_lenet_epoch.pth"
model = LeNet() # 定义LeNet模型
criterion = nn.CrossEntropyLoss() # 定义损失函数
model.load_state_dict(torch.load(model_path)) # 载入模型和参数
model.eval() # 切换模型到测试状态

test_loss = 0
correct = 0
total = 0
with torch.no_grad(): # 关闭计算图
    for batch_idx, (inputs, targets) in enumerate(data_test_loader):

        outputs = model(inputs)
        loss = criterion(outputs, targets)

        test_loss += loss.item()
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += predicted.eq(targets).sum().item()

    print(batch_idx, len(data_test_loader), 'Loss: %.3f | Acc: %.3f%% (%d/%d)'
          % (test_loss/(batch_idx+1), 100.*correct/total, correct, total))
```

```
0 10 Loss: 0.105 | Acc: 96.777% (991/1024)
1 10 Loss: 0.139 | Acc: 95.654% (1959/2048)
2 10 Loss: 0.141 | Acc: 95.605% (2937/3072)
3 10 Loss: 0.147 | Acc: 95.239% (3901/4096)
4 10 Loss: 0.142 | Acc: 95.312% (4880/5120)
5 10 Loss: 0.132 | Acc: 95.638% (5876/6144)
6 10 Loss: 0.121 | Acc: 96.010% (6882/7168)
7 10 Loss: 0.111 | Acc: 96.362% (7894/8192)
8 10 Loss: 0.104 | Acc: 96.615% (8904/9216)
9 10 Loss: 0.106 | Acc: 96.560% (9656/10000)
```

【注意】：由于在测试状态下不需要使用计算图的功能，因此，这里使用torch.no_grad()上下文管理器，只进行简单的模型推断计算。

接下来的流程和 train.py的内容类似，只不过使用的数据载入器是data_test_loader。

最后输出的结果是保存的模型在测试集的损失函数和准确率。

由于测试数据和训练数据不同，只需要进行一次迭代期的计算。

5. 超参数的修改和argparse库的使用（选学）

在神经网络的构建过程中，常常需要修改神经网络的超参数（Hyper Parameters),比如，迷你批次的大小、学习率等。

为了方便参数的修改，可以通过命令行的方式来传递这些超参数。

在这一节的最后，我们将介绍如何使用argparse库来处理Python脚本的参数，方便对深度学习超参数的调优。

使用argparse库指定LeNet神经网络超参数的方法，代码如下：

```
import argparse

parser=argparse.ArgumentParser(description='PyTorch LeNet Training')

parser.add_argument('--lr', default=0.01,type=float,help='Learning rate')

parser.add_argument('--batch-size', '-b', default=256,type=int,help='Batchsize')

parser.add_argument('--epoch', default=4,type=int,help='Epoch')
```

```
args=parser.parse_args()
```

在代码中，首先使用`argparse.ArgumentParser`类来构造一个命令行参数处理器的实例，`description`参数用来描述具体命令行处理器的作用。

可以使用

```
python prog.py -h
```

打印出帮助内容，其内容与代码中具体每个选项的`help`参数传入的字符串有关。

在后续的代码中，添加了三个参数，`--lr`是一个浮点类型，代表学习率；`--batch-size`是一个整数类型，代表迷你批次的大小；`--epoch`是一个整数类型，代表训练的迭代期。

可以分别通过 `args.lr`、`args.batch_size`和`args.epoch`来访问这三个变量

（注意，在命令行里，参数中间包含的连字符 `-` 变成了Python中变量的下划线`_`，而参数前面的两个连字符`--`在转换成变量的时候会被自动过滤掉）。

大家可以尝试在 `train.py`和 `inference.py`中加入各种超参数的选项来满足深度学习工程中调参的需求。

调用时在cmd中输入命令如下示例：

```
python train_args.py --batch-size=128 --epoch=10
```

前提是已经在`train_args.py`文件中加入了这些超参数。`train_args.py`代码如下：【注：此代码不能在jupyter notebook中直接使用】

```

In [ ]: # 该代码在LeNet模型的训练过程中加入了argparse库的使用

import torch
import torch.nn as nn
import torch.optim as optim
import argparse

from torchvision.datasets import MNIST
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

parser=argparse.ArgumentParser(description='PyTorch LeNet Training')

parser.add_argument('--lr', default=0.01, type=float, help='Learning rate')

parser.add_argument('--batch-size', '-b', default=256, type=int, help='Batchsize')

parser.add_argument('--epoch', default=4, type=int, help='Epoch')

args=parser.parse_args()

batch_size = args.batch_size
lr = args.lr # 定义学习率
epochs = args.epoch # 定义训练迭代期
# 训练数据载入器
data_train = MNIST('./data',
                    download=True,
                    transform=transforms.Compose([
                        transforms.Resize((32, 32)),
                        transforms.ToTensor()]))
data_test = MNIST('./data',
                  train=False,
                  download=True,
                  transform=transforms.Compose([
                        transforms.Resize((32, 32)),
                        transforms.ToTensor()]))
data_train_loader = DataLoader(data_train, batch_size=batch_size,
                               shuffle=True, num_workers=2)
data_test_loader = DataLoader(data_test, batch_size=batch_size,
                              num_workers=2)

class LeNet(nn.Module):

    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 3)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 3)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc3 = nn.Linear(16*6*6, 120)
        self.fc4 = nn.Linear(120, 84)
        self.fc5 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool1(torch.relu(self.conv1(x)))
        x = self.pool2(torch.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = torch.relu(self.fc3(x))
        x = torch.relu(self.fc4(x))
        x = self.fc5(x)
        return x

def train():
    model = LeNet() # 定义LeNet模型
    model.train() # 切换模型到训练状态

    criterion = nn.CrossEntropyLoss() # 定义损失函数
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9,
                           weight_decay=5e-4) # 定义随机梯度下降优化器

    for epoch in range(epochs):
        train_loss = 0
        correct = 0
        total = 0
        for batch_idx, (inputs, targets) in enumerate(data_train_loader):

```

```

optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, targets)
loss.backward()
optimizer.step()

train_loss += loss.item()
_, predicted = outputs.max(1)
total += targets.size(0)
correct += predicted.eq(targets).sum().item()

if batch_idx % 50 == 0 or batch_idx == (len(data_train_loader) - 1):
    print(epoch, batch_idx, len(data_train_loader), 'Loss: %.3f | Acc: %.3f%% (%d/%d)'
          % (train_loss / (batch_idx + 1), 100. * correct / total, correct, total))

# 模型的保存
torch.save(model.state_dict(), 'model_lenet_epoch.pth') # 模型的状态字典

if __name__ == "__main__":
    train()

```

参考文献:

张校捷《深入浅出PyTorch：从模型到源码》电子工业出版社，2020.